

# C# OOP

## QUAN HỆ GIỮA CÁC OBJECT

### Association - Aggregation - Composition

**Phần 7 trong lộ trình OOP C#**

Lý thuyết đầy đủ - Ví dụ C# - Phân tích thiết kế - Bài tập - Đáp án

#### PHẠM VI

Tài liệu này tiếp nối các kiến thức Class/Object, Encapsulation, Inheritance, Polymorphism, Abstraction và Interface. Trọng tâm mới là quan hệ has-a giữa các object. Không đi sâu vào Generic, LINQ, SOLID hoặc Dependency Injection.

*Tài liệu học tập cá nhân - C# theo hướng .NET Backend*

## Mục lục

| Mục | Nội dung                                               |
|-----|--------------------------------------------------------|
| 1   | Bản đồ kiến thức: từ is-a sang has-a                   |
| 2   | is-a và has-a                                          |
| 3   | Association - quan hệ liên kết/sử dụng                 |
| 4   | Aggregation - has-a dạng lỏng                          |
| 5   | Composition - has-a dạng chặt                          |
| 6   | So sánh Association, Aggregation, Composition          |
| 7   | Quan hệ một-một và một-nhiều trong code                |
| 8   | Khi nào dùng Inheritance, khi nào dùng Composition     |
| 9   | Ví dụ tổng hợp: Order - Customer - Product - OrderItem |
| 10  | Lỗi thiết kế và lỗi code thường gặp                    |
| 11  | Bài tập thực hành                                      |
| 12  | Đáp án và hướng giải                                   |
| 13  | Cheat sheet và checklist tự đánh giá                   |

## Cách học với tài liệu này

1. Đọc định nghĩa và cố gắng tự nói lại bằng lời của bạn.
2. Gõ lại ví dụ nhỏ, không chỉ đọc code.
3. Ở các bảng so sánh, tập trung vào vòng đời object và nơi object được tạo.
4. Làm bài tập trước khi xem chương đáp án.
5. Cuối mỗi chương chỉ cần đọc phần “Tóm tắt cần nhớ” để ôn nhanh.

# 1. Bản đồ kiến thức: từ is-a sang has-a

## 1.1 Bạn đang đứng ở đâu trong OOP?

Ở các phần trước, chúng ta đã xây dựng các class như Employee, OfficeEmployee, SalesEmployee, Shape, Circle, Rectangle và IBonusEligible. Phần lớn các ví dụ đó tập trung vào quan hệ kế thừa và đa hình.

**Sơ đồ quan hệ kế thừa đã học**

```
Employee
|
+-- OfficeEmployee
+-- SalesEmployee
+-- PartTimeEmployee

Shape
|
+-- Circle
+-- Rectangle
```

Các quan hệ trên đều trả lời câu hỏi: “Đối tượng này có phải là một dạng của đối tượng kia không?”. Đây là cách kiểm tra quan hệ is-a.

### VÍ DỤ

OfficeEmployee is-an Employee.  
Circle is-a Shape.  
Student is-a Person.

## 1.2 Vấn đề mới: object cần object khác để hoạt động

Trong chương trình thực tế, một object thường không chỉ “là một loại của” object khác. Nó còn chứa, sử dụng hoặc liên kết với nhiều object khác.

```
Car has an Engine.
Order has a Customer.
Order has many OrderItems.
Department has Employees.
Teacher teaches Student.
```

Những trường hợp này không nên giải quyết bằng kế thừa. Chúng dẫn chúng ta đến nhóm quan hệ object relationship, thường được diễn giải bằng ý tưởng has-a hoặc uses-a.

## 1.3 Mục tiêu của phần 7

- Phân biệt chính xác is-a và has-a.
- Hiểu Association, Aggregation và Composition không chỉ bằng định nghĩa mà bằng vòng đời object.
- Biết biểu diễn quan hệ một-một và một-nhiều bằng property và List<T>.
- Biết khi nào không nên kế thừa chỉ để tái sử dụng code.
- Chuẩn bị tư duy cho Entity Framework Core và mô hình dữ liệu sau này.

## Tóm tắt cần nhớ

### GHI NHỚ

- Inheritance giải quyết quan hệ is-a.
- Object relationship giải quyết việc object chứa, biết hoặc sử dụng object khác.
- Association, Aggregation và Composition đều liên quan đến quan hệ giữa các object, nhưng mức độ gắn kết khác nhau.

**Một câu cần nhớ:** Sau khi học inheritance, bước tiếp theo không phải là kế thừa nhiều hơn, mà là học cách các object phối hợp với nhau.

## 2. is-a và has-a

### 2.1 Quan hệ is-a

is-a là cách diễn đạt quan hệ kế thừa. Nếu B kế thừa A thì B phải có thể được xem là một A về mặt ý nghĩa nghiệp vụ.

```
public abstract class Employee
{
    public string FullName { get; }
}

public class OfficeEmployee : Employee
{
}
```

Ta đọc: OfficeEmployee is-an Employee. Đây là quan hệ hợp lý vì nhân viên văn phòng thực sự là một loại nhân viên.

#### CÂU HỎI KIỂM TRA

Nếu bạn không thể đọc câu “B là một A” một cách tự nhiên, rất có thể kế thừa không phải lựa chọn phù hợp.

### 2.2 Quan hệ has-a

has-a diễn tả một object có, chứa hoặc quản lý một object khác. Trong code, quan hệ này thường xuất hiện khi một property hoặc field có kiểu là một class khác.

```
public class Engine
{
    public string Model { get; }
}

public class Car
{
    public Engine Engine { get; }
}
```

Ta đọc: Car has-an Engine. Car không phải là Engine, do đó không nên viết Car : Engine.

### 2.3 So sánh bằng câu hỏi thiết kế

| Câu hỏi                             | Nếu trả lời “có”                 | Ví dụ                         |
|-------------------------------------|----------------------------------|-------------------------------|
| B có phải là một A không?           | Cần nhắc inheritance             | OfficeEmployee is-an Employee |
| A có/chứa B không?                  | Cần nhắc composition/aggregation | Car has-an Engine             |
| A chỉ sử dụng hoặc tương tác với B? | Cần nhắc association             | Teacher teaches Student       |

## 2.4 Một lỗi rất phổ biến: kế thừa chỉ để lấy code

```
public class Engine
{
    public void Start() { }
}

public class Car : Engine    // Sai về thiết kế
{
}
```

Có thể người viết chỉ muốn Car sử dụng Start(), nhưng câu “Car is-an Engine” là sai về mặt mô hình. Tái sử dụng code không đủ để chứng minh quan hệ kế thừa.

### Thiết kế hợp lý hơn

```
public class Car
{
    public Engine Engine { get; }

    public Car(Engine engine)
    {
        Engine = engine;
    }
}
```

## Tóm tắt cần nhớ

### GHI NHỚ

- is-a -> inheritance.
- has-a -> object chứa hoặc quản lý object khác.
- uses-a -> object sử dụng object khác trong một hành vi.
- Không dùng inheritance chỉ vì muốn dùng lại method.

**Một câu cần nhớ:** Nếu mối quan hệ không đọc được là “B là một A”, hãy kiểm tra has-a trước khi nghĩ đến inheritance.

## 3. Association - quan hệ liên kết hoặc sử dụng

### 3.1 Định nghĩa

Association là quan hệ tổng quát giữa hai object khi chúng biết nhau, sử dụng nhau hoặc tương tác với nhau để thực hiện một nhiệm vụ. Association không nhất thiết thể hiện quyền sở hữu và cũng không nhất thiết ràng buộc vòng đời của hai object.

#### Ý CHÍNH

Association trả lời: “Hai object có liên hệ hoặc cần tương tác với nhau hay không?”

### 3.2 Association thông qua parameter

```
public class Student
{
    public string Name { get; }

    public Student(string name)
    {
        Name = name;
    }
}

public class Teacher
{
    public string Name { get; }

    public Teacher(string name)
    {
        Name = name;
    }

    public void Teach(Student student)
    {
        Console.WriteLine(
            $"{Name} is teaching {student.Name}");
    }
}
```

Ở đây Teacher không lưu Student như một thành phần bên trong. Student chỉ được truyền vào method Teach() khi cần. Teacher và Student đều có vòng đời độc lập.

```
Student student = new Student("An");
Teacher teacher = new Teacher("Hung");

teacher.Teach(student);
```

### 3.3 Association thông qua property

Association cũng có thể được biểu diễn bằng property nếu một object cần ghi nhớ object mà nó đang liên kết.

```
public class Doctor
{
    public string Name { get; }
```

```

    public Patient? CurrentPatient { get; private set; }

    public Doctor(string name)
    {
        Name = name;
    }

    public void Examine(Patient patient)
    {
        CurrentPatient = patient;
    }
}

```

Điều quan trọng vẫn là: Patient có thể tồn tại trước Doctor, sau Doctor hoặc được sử dụng bởi nhiều Doctor khác. Quan hệ không khẳng định Patient “thuộc sở hữu vòng đời” của Doctor.

### 3.4 Association một chiều và hai chiều

Association một chiều nghĩa là A biết B nhưng B không cần biết A. Association hai chiều nghĩa là cả hai object giữ tham chiếu tới nhau. Hai chiều có thể cần thiết, nhưng làm thiết kế phức tạp hơn và dễ tạo vòng tham chiếu.

```

// Một chiều
class Order
{
    public Customer Customer { get; }
}

// Hai chiều (chỉ dùng khi thật sự cần)
class Customer
{
    public List<Order> Orders { get; }
}

class Order
{
    public Customer Customer { get; }
}

```

#### LỖI THƯỜNG GẶP

Thấy hai class có liên quan không có nghĩa là phải cho cả hai giữ property của nhau. Chỉ thêm hướng liên kết mà nghiệp vụ cần.

### Tóm tắt cần nhớ

#### GHI NHỚ

- Association là quan hệ liên kết/sử dụng tổng quát.
- Không nhấn mạnh quyền sở hữu vòng đời.
- Có thể thể hiện qua parameter, property hoặc collection.
- Có thể một chiều hoặc hai chiều.

**Một câu cần nhớ:** Association nói về việc hai object “có liên hệ với nhau”, chưa nói object nào sở hữu object nào.



## 4. Aggregation - quan hệ has-a dạng lỏng

### 4.1 Định nghĩa

Aggregation là một dạng quan hệ has-a trong đó object chứa giữ hoặc quản lý một hay nhiều object thành phần, nhưng các object thành phần vẫn có thể tồn tại độc lập. Nói cách khác, object thành phần không phụ thuộc hoàn toàn vào vòng đời của object chứa.

#### Ý CHÍNH

Aggregation = “có”, nhưng không “sở hữu vòng đời tuyệt đối”.

### 4.2 Dấu hiệu nhận biết trong code

Một dấu hiệu thường gặp là object thành phần được tạo bên ngoài, sau đó được truyền vào constructor hoặc method của object chứa.

```
public class Student
{
    public string Id { get; }
    public string FullName { get; }

    public Student(string id, string fullName)
    {
        Id = id;
        FullName = fullName;
    }
}

public class Course
{
    public string Name { get; }
    public List<Student> Students { get; }

    public Course(string name, List<Student> students)
    {
        Name = name;
        Students = students;
    }
}
```

#### Cách tạo object

```
Student s1 = new Student("SV01", "An");
Student s2 = new Student("SV02", "Binh");

Course course = new Course(
    "C# OOP",
    new List<Student> { s1, s2 });
```

s1 và s2 tồn tại trước khi Course được tạo. Nếu biến course không còn được sử dụng, hai Student vẫn có thể được tham chiếu ở nơi khác. Đó là ý nghĩa độc lập về vòng đời trong mô hình.

### 4.3 Ví dụ Department - Employee

```
public class Department
{
    ...
}
```

```

public string Name { get; }
private readonly List<Employee> _employees = new();

public IReadOnlyList<Employee> Employees => _employees;

public Department(string name)
{
    Name = name;
}

public void AddEmployee(Employee employee)
{
    if (employee is null)
        throw new ArgumentNullException(nameof(employee));

    _employees.Add(employee);
}
}

```

Employee có thể chuyển phòng ban, tồn tại trước Department hoặc sau khi một Department bị thay đổi. Vì vậy Department - Employee thường được dùng để minh họa aggregation.

#### 4.4 Không nên hiểu Aggregation chỉ bằng “new ở ngoài”

“Object được new ở ngoài” là dấu hiệu học tập dễ quan sát, nhưng bản chất quan trọng hơn là vòng đời và quyền sở hữu. Trong code thực tế, dependency injection hoặc factory có thể làm vị trí new thay đổi. Hãy tập trung vào câu hỏi: object thành phần có ý nghĩa độc lập với object chứa không?

##### GHI NHỚ NÂNG CAO

Vị trí từ khóa new giúp người mới nhận biết quan hệ, nhưng không phải định nghĩa chính thức duy nhất. Bản chất là mức độ sở hữu và vòng đời object.

#### Tóm tắt cần nhớ

##### GHI NHỚ

- Aggregation là has-a dạng lỏng.
- Object thành phần có thể tồn tại độc lập.
- Object thành phần thường được tạo bên ngoài rồi truyền vào.
- Một Department có Employees; Employee vẫn là object độc lập.

**Một câu cần nhớ:** Aggregation = chứa/sử dụng object khác nhưng không buộc vòng đời của object đó vào object chứa.

## 5. Composition - quan hệ has-a dạng chặt

### 5.1 Định nghĩa

Composition là quan hệ has-a mạnh hơn aggregation. Object thành phần được xem là một bộ phận cấu thành của object chứa. Về mặt mô hình, vòng đời hoặc ý nghĩa của object thành phần gắn chặt với object chứa hơn.

#### Ý CHÍNH

Composition = object thành phần là “một phần của” object chứa và thường do object chứa tạo/quản lý.

### 5.2 Ví dụ Computer - CPU

```
public class CPU
{
    public string Model { get; }
    public int CoreCount { get; }

    public CPU(string model, int coreCount)
    {
        if (string.IsNullOrEmpty(model))
            throw new ArgumentException(
                "Model không được để trống",
                nameof(model));

        if (coreCount <= 0)
            throw new ArgumentException(
                "Số nhân phải lớn hơn 0",
                nameof(coreCount));

        Model = model.Trim();
        CoreCount = coreCount;
    }
}

public class Computer
{
    public string Name { get; }
    public CPU CPU { get; }

    public Computer(
        string name,
        string cpuModel,
        int coreCount)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException(
                "Tên máy không được để trống",
                nameof(name));

        Name = name.Trim();
        CPU = new CPU(cpuModel, coreCount);
    }
}
```

```
Computer computer = new Computer(
    "Gaming PC",
    "Ryzen 7",
    8);
```

Code bên ngoài không cần tự tạo CPU. Computer chịu trách nhiệm tạo CPU cho chính nó. Đây là ví dụ composition rõ ràng ở mức học hiện tại.

### 5.3 Ví dụ Order - OrderItem

OrderItem thường chỉ có ý nghĩa trong một Order cụ thể. Một dòng chi tiết đơn hàng không phải là đơn hàng, nhưng là một phần cấu thành của đơn hàng.

```
public class OrderItem
{
    public string ProductName { get; }
    public int Quantity { get; }
    public decimal UnitPrice { get; }

    public decimal SubTotal => Quantity * UnitPrice;

    public OrderItem(
        string productName,
        int quantity,
        decimal unitPrice)
    {
        ProductName = productName;
        Quantity = quantity;
        UnitPrice = unitPrice;
    }
}

public class Order
{
    private readonly List<OrderItem> _items = new();

    public IReadOnlyList<OrderItem> Items => _items;

    public void AddItem(
        string productName,
        int quantity,
        decimal unitPrice)
    {
        _items.Add(new OrderItem(
            productName,
            quantity,
            unitPrice));
    }
}
```

Order tự tạo OrderItem thông qua AddItem(). Điều này thể hiện quyền quản lý chặt hơn so với việc bên ngoài tự ý tạo và gắn bất kỳ OrderItem nào.

### 5.4 Composition không có nghĩa là object phải “biến mất khỏi bộ nhớ” ngay

Trong C#, bộ nhớ được quản lý bởi garbage collector. Khi nói vòng đời gắn chặt trong composition, ta đang nói về vòng đời và ý nghĩa trong mô hình nghiệp vụ, không phải cam kết rằng object con bị hủy vật lý ngay khi object cha mất tham chiếu.

#### QUAN TRỌNG

Đừng nhầm “vòng đời nghiệp vụ” với thời điểm Garbage Collector thu hồi bộ nhớ.

## Tóm tắt cần nhớ

### GHI NHỚ

- Composition là has-a dạng chặt.
- Object thành phần thường được object chứa tạo và quản lý.
- Object con thường có ý nghĩa như một phần của object cha.
- Ví dụ: Computer - CPU, Order - OrderItem.

**Một câu cần nhớ:** Composition = object chứa không chỉ “biết” object con, mà còn chịu trách nhiệm mạnh hơn đối với việc tạo/quản lý nó.

## 6. So sánh Association, Aggregation và Composition

### 6.1 Bảng so sánh trọng tâm

| Tiêu chí                   | Association        | Aggregation                      | Composition               |
|----------------------------|--------------------|----------------------------------|---------------------------|
| Ý nghĩa                    | Liên kết/sử dụng   | Has-a dạng lỏng                  | Has-a dạng chặt           |
| Sở hữu                     | Không nhấn mạnh    | Có quản lý nhưng không tuyệt đối | Quản lý mạnh hơn          |
| Vòng đời object thành phần | Độc lập            | Độc lập                          | Gắn chặt hơn về nghiệp vụ |
| Cách tạo thường gặp        | Parameter/property | Tạo ngoài rồi truyền vào         | Object chứa tự tạo        |
| Ví dụ                      | Teacher - Student  | Department - Employee            | Order - OrderItem         |

### 6.2 Cùng một cặp class có thể được thiết kế khác nhau

Một điểm quan trọng: tên hai class không tự động quyết định quan hệ. Quan hệ phụ thuộc vào nghiệp vụ và cách bạn muốn quản lý vòng đời.

Ví dụ Engine - Car có thể được dạy như composition nếu engine là bộ phận riêng của chiếc xe. Nhưng trong một hệ thống quản lý kho động cơ, Engine có thể tồn tại độc lập trước khi được lắp vào Car. Khi đó quan hệ có thể gần aggregation hơn.

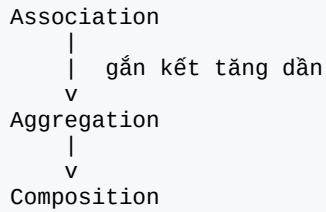
#### KHÔNG HỌC THUỘC MÁY MÓC

Đừng chỉ ghi nhớ “Car - Engine = Composition”. Hãy hỏi: Engine có tồn tại độc lập trong bài toán này không? Ai tạo? Ai quản lý? Có thể chuyển sang Car khác không?

### 6.3 Cây quyết định đơn giản

```
B có phải là một A?  
|  
+-- Có -> Inheritance (is-a)  
|  
+-- Không  
|  
+-- A có/chứa B?  
|  
|   +-- B độc lập mạnh -> Aggregation  
|   +-- B gắn chặt -> Composition  
|  
+-- A chỉ tương tác với B -> Association
```

## 6.4 Mức độ gắn kết



Sơ đồ trên giúp nhớ mức độ, nhưng trong thiết kế thực tế ranh giới có thể không tuyệt đối. Điều quan trọng là bạn mô tả đúng nghiệp vụ và kiểm soát object hợp lý.

### Tóm tắt cần nhớ

#### GHÌ NHỚ

- Association là khái niệm rộng nhất.
- Aggregation và Composition đều là các cách mô tả has-a với mức sở hữu khác nhau.
- Không xác định quan hệ chỉ bằng tên class; phải dựa vào nghiệp vụ và vòng đời.
- Composition thường chặt hơn Aggregation.

**Một câu cần nhớ:** Muốn phân biệt Aggregation và Composition, hãy hỏi về tính độc lập của object thành phần và trách nhiệm quản lý vòng đời.

## 7. Quan hệ một-một và một-nhiều trong code

### 7.1 Quan hệ một-một

Một object A liên kết với một object B. Trong C#, cách biểu diễn đơn giản là một property có kiểu class B.

```
public class Address
{
    public string City { get; }

    public Address(string city)
    {
        City = city;
    }
}

public class Student
{
    public string FullName { get; }
    public Address Address { get; }

    public Student(
        string fullName,
        Address address)
    {
        FullName = fullName;
        Address = address;
    }
}
```

Một Student có một Address. Đây là quan hệ một-một ở mức mô hình class. Nó có thể là association, aggregation hoặc composition tùy quy tắc vòng đời.

### 7.2 Quan hệ một-nhiều

Một object A liên kết với nhiều object B. Trong C#, chúng ta thường biểu diễn bằng List<B> hoặc một collection tương tự.

```
public class Department
{
    public string Name { get; }
    public List<Employee> Employees { get; }

    public Department(string name)
    {
        Name = name;
        Employees = new List<Employee>();
    }
}
```

### 7.3 Encapsulation cho collection

Nếu expose List<T> trực tiếp, code bên ngoài có thể tự do Add, Remove hoặc Clear mà không đi qua quy tắc nghiệp vụ của class.

```
department.Employees.Clear(); // Có thể phá quy tắc nghiệp vụ
```

Một thiết kế chặt hơn là giữ List riêng và chỉ expose IReadOnlyList<T>.



```

public class Department
{
    private readonly List<Employee> _employees = new();

    public IReadOnlyList<Employee> Employees => _employees;

    public void AddEmployee(Employee employee)
    {
        if (employee is null)
            throw new ArgumentNullException(nameof(employee));

        _employees.Add(employee);
    }
}

```

Cách này kết hợp kiến thức quan hệ object với Encapsulation mà bạn đã học trước.

## 7.4 Liên hệ với database/EF Core

Sau này khi học Entity Framework Core, các property tham chiếu và collection như Customer, Order, OrderItems sẽ trở thành navigation properties. Vì vậy hiểu quan hệ object ngay bây giờ sẽ giúp bạn học quan hệ database dễ hơn.

| Mô hình object        | Ý nghĩa gần với database                     |
|-----------------------|----------------------------------------------|
| Customer Customer     | Nhiều bản ghi có thể tham chiếu một Customer |
| List<Order> Orders    | Một Customer có nhiều Order                  |
| List<OrderItem> Items | Một Order có nhiều dòng chi tiết             |

## Tóm tắt cần nhớ

### GHI NHỚ

- Một-một thường dùng một property object.
- Một-nhiều thường dùng List<T> hoặc collection.
- Collection cũng cần encapsulation.
- Các quan hệ object là nền tảng để hiểu navigation property trong EF Core.

**Một câu cần nhớ:** Một property biểu diễn “một”; một collection biểu diễn “nhiều”.

## 8. Khi nào dùng Inheritance, khi nào dùng Composition

### 8.1 Inheritance phù hợp khi nào?

- Có quan hệ is-a rõ ràng.
- Lớp con thực sự có thể thay thế lớp cha trong ngữ cảnh chung.
- Có trạng thái/hành vi chung hợp lý để đặt ở lớp cha.
- Cần polymorphism qua kiểu cha.

```
Shape shape = new Circle(5);  
Console.WriteLine(shape.CalculateArea());
```

### 8.2 Composition phù hợp khi nào?

- Object cần nhiều thành phần khác nhau để hoạt động.
- Quan hệ là has-a chứ không phải is-a.
- Muốn thay đổi hoặc kết hợp các thành phần linh hoạt hơn.
- Muốn tránh cây kế thừa quá sâu hoặc kế thừa sai ý nghĩa.

```
public class Car  
{  
    public Engine Engine { get; }  
    public GPS GPS { get; }  
    public Radio Radio { get; }  
}
```

### 8.3 Liên hệ với câu hỏi đa kế thừa class

C# không hỗ trợ một class kế thừa trực tiếp nhiều class. Thay vì cố mô phỏng “Car : Engine, GPS, Radio”, ta thường kết hợp một base class nếu thật sự có quan hệ is-a, nhiều interface cho khả năng, và composition cho các thành phần.

```
public class SmartCar : Vehicle, ITrackable, IConnectable  
{  
    public Engine Engine { get; }  
    public GPS GPS { get; }  
}
```

### CÔNG THỨC THIẾT KẾ DỄ NHỚ

1 base class nếu có is-a + nhiều interface nếu có nhiều khả năng + composition cho các thành phần has-a.

### 8.4 Composition over inheritance

Trong thiết kế phần mềm, “ưu tiên composition hơn inheritance” thường được dùng như một nguyên tắc định hướng. Nó không có nghĩa inheritance là xấu; nghĩa là đừng chọn inheritance nếu mục tiêu chỉ là tái sử dụng code hoặc ghép nhiều khả năng không cùng bản chất.

| Chọn        | Khi                                                |
|-------------|----------------------------------------------------|
| Inheritance | B thật sự là một A; cần polymorphism qua A         |
| Interface   | Nhiều class khác nhau cùng có một khả năng/hành vi |
| Composition | A cần có/chứa/điều phối B                          |

## Tóm tắt cần nhớ

### GHI NHỚ

- Inheritance cho is-a; composition cho has-a.
- C# dùng 1 base class + nhiều interface + composition thay vì đa kế thừa class.
- Không kế thừa chỉ để tái sử dụng code.
- Composition giúp ghép nhiều thành phần linh hoạt.

**Một câu cần nhớ:** Nếu bạn chỉ muốn “có một thứ để dùng”, hãy nghĩ composition trước khi nghĩ inheritance.

## 9. Ví dụ tổng hợp: Order - Customer - Product - OrderItem

### 9.1 Phân tích nghiệp vụ

Ta thiết kế một mô hình bán hàng nhỏ. Mục tiêu không phải tạo ứng dụng hoàn chỉnh, mà dùng một ví dụ để thấy nhiều quan hệ object xuất hiện cùng lúc.

```
Customer 1 ---- n Order
Order    1 ---- n OrderItem
Product  1 ---- n OrderItem
```

Một Customer có thể có nhiều Order. Một Order có nhiều OrderItem. Mỗi OrderItem tham chiếu một Product.

### 9.2 Product

```
public class Product
{
    public string Id { get; }
    public string Name { get; }
    public decimal Price { get; private set; }

    public Product(string id, string name, decimal price)
    {
        if (string.IsNullOrEmpty(id))
            throw new ArgumentException(nameof(id));

        if (string.IsNullOrEmpty(name))
            throw new ArgumentException(nameof(name));

        if (price <= 0)
            throw new ArgumentException(nameof(price));

        Id = id.Trim();
        Name = name.Trim();
        Price = price;
    }
}
```

### 9.3 Customer

```
public class Customer
{
    public string Id { get; }
    public string FullName { get; }

    public Customer(string id, string fullName)
    {
        Id = id;
        FullName = fullName;
    }
}
```

### 9.4 OrderItem

```
public class OrderItem
{
```

```

public Product Product { get; }
public int Quantity { get; }
public decimal UnitPrice { get; }

public decimal SubTotal => Quantity * UnitPrice;

public OrderItem(Product product, int quantity)
{
    if (product is null)
        throw new ArgumentNullException(nameof(product));

    if (quantity <= 0)
        throw new ArgumentException(nameof(quantity));

    Product = product;
    Quantity = quantity;
    UnitPrice = product.Price;
}
}

```

OrderItem không kế thừa Product. Nó “có một Product” bằng property Product. Đây là has-a.

## 9.5 Order

```

public class Order
{
    private readonly List<OrderItem> _items = new();

    public string Id { get; }
    public Customer Customer { get; }
    public IReadOnlyList<OrderItem> Items => _items;

    public Order(string id, Customer customer)
    {
        if (customer is null)
            throw new ArgumentNullException(nameof(customer));

        Id = id;
        Customer = customer;
    }

    public void AddItem(Product product, int quantity)
    {
        _items.Add(new OrderItem(product, quantity));
    }

    public decimal CalculateTotal()
    {
        decimal total = 0;

        foreach (OrderItem item in _items)
        {
            total += item.SubTotal;
        }

        return total;
    }
}

```

## 9.6 Sử dụng

```
Customer customer = new Customer("KH01", "Nguyen Van A");
Product p1 = new Product("SP01", "Keyboard", 500_000m);
Product p2 = new Product("SP02", "Mouse", 300_000m);

Order order = new Order("HD01", customer);
order.AddItem(p1, 2);
order.AddItem(p2, 1);

Console.WriteLine(order.CalculateTotal());
```

## 9.7 Phân loại các quan hệ

| Quan hệ             | Cách nhìn                                                                          |
|---------------------|------------------------------------------------------------------------------------|
| Order - Customer    | Order tham chiếu Customer; Customer tồn tại độc lập -> gần aggregation/association |
| Order - OrderItem   | Order tạo và quản lý OrderItem -> composition rõ hơn                               |
| OrderItem - Product | OrderItem tham chiếu Product; Product tồn tại độc lập -> aggregation/association   |

### BÀI HỌC

Trong một hệ thống thực tế, một class có thể đồng thời tham gia nhiều kiểu quan hệ với các class khác nhau.

## Tóm tắt cần nhớ

### GHI NHỚ

- Không phải mọi quan hệ trong hệ thống đều là inheritance.
- Order có Customer, có nhiều OrderItem; OrderItem có Product.
- Collection biểu diễn one-to-many.
- Encapsulation vẫn cần được áp dụng cho collection và trạng thái.

**Một câu cần nhớ:** Thiết kế OOP tốt là chọn đúng quan hệ cho từng cặp object, không cố ép mọi thứ vào một cây kế thừa.

## 10. Lỗi thiết kế và lỗi code thường gặp

### 10.1 Nhầm is-a với has-a

```
public class Car : Engine    // Sai ý nghĩa
{
}
```

Sửa bằng property:

```
public class Car
{
    public Engine Engine { get; }
}
```

### 10.2 Nhầm has-a với interface

has-a không đồng nghĩa với interface. Interface mô tả khả năng/hành vi; has-a mô tả object chứa hoặc tham chiếu object khác.

```
class Printer : IPrintable    // khả năng
class Computer { CPU CPU { get; } } // has-a
```

### 10.3 Phân loại Aggregation/Composition chỉ dựa trên “new nằm đâu”

Vị trí new là dấu hiệu dễ học nhưng không phải toàn bộ bản chất. Hãy xem object con có ý nghĩa độc lập hay không và ai chịu trách nhiệm quản lý nó.

### 10.4 Public List<T> cho phép bên ngoài phá trạng thái

```
public List<OrderItem> Items { get; set; } // quá mở
```

Nên cân nhắc private collection + IReadOnlyList<T> + method Add/Remove có validation.

### 10.5 Tạo quan hệ hai chiều không cần thiết

Nếu Order đã biết Customer, chưa chắc Customer phải có List<Order>. Hai chiều chỉ thêm khi nghiệp vụ thực sự cần điều hướng cả hai phía.

### 10.6 Quên validation object parameter

```
public Order(string id, Customer customer)
{
    Customer = customer; // có thể null
}
```

Nên kiểm tra null trước khi lưu object tham chiếu.

### 10.7 Kế thừa quá sâu

```
A -> B -> C -> D -> E
```

Cây kế thừa sâu làm thay đổi ở lớp trên lan xuống nhiều lớp dưới. Nếu các lớp chỉ đang ghép các khả năng/thành phần, composition/interface thường dễ bảo trì hơn.

## Tóm tắt cần nhớ

### GHI NHỚ

- Không nhầm is-a với has-a.
- Interface không phải has-a.
- Aggregation/Composition phải nhìn vòng đời và ownership.
- Collection cần encapsulation.
- Tránh liên kết hai chiều và cây kế thừa sâu nếu không cần.

**Một câu cần nhớ:** Một thiết kế chạy được chưa chắc là một thiết kế đúng ý nghĩa OOP.



## 11. Bài tập thực hành

### 11.1 Bài 1 - Composition: Computer và CPU

#### YÊU CẦU

Tạo CPU có Model, CoreCount. Tạo Computer có Name và CPU. Khi tạo Computer, Computer phải tự tạo CPU bên trong constructor. Có validation phù hợp.

#### Kết quả sử dụng mong muốn

```
Computer computer = new Computer(  
    "Gaming PC",  
    "Ryzen 7",  
    8);
```

### 11.2 Bài 2 - Aggregation: Course và Student

#### YÊU CẦU

Tạo Student ở bên ngoài trước. Course nhận List<Student> hoặc có method AddStudent(Student). Student vẫn phải có thể tồn tại độc lập nếu không còn Course.

### 11.3 Bài 3 - Association: Doctor và Patient

#### YÊU CẦU

Doctor có method Examine(Patient patient). Patient không phải thành phần sở hữu của Doctor. In ra tên bác sĩ và bệnh nhân đang được khám.

### 11.4 Bài 4 - Phân loại quan hệ

6. Dog - Animal
7. Car - Engine
8. Department - Employee
9. Doctor - Patient
10. Order - OrderItem
11. Invoice - IPrintable
12. Student - Address

Với mỗi cặp, hãy chọn is-a, Association, Aggregation hoặc Composition và giải thích bằng 1-2 câu. Nếu thấy phụ thuộc nghiệp vụ, hãy nói điều kiện nào làm quan hệ thay đổi.

### 11.5 Bài 5 - Encapsulation cho collection

#### YÊU CẦU

Thiết kế Team có nhiều Member. Không cho code bên ngoài Clear() trực tiếp collection. Chỉ cho thêm

thành viên qua AddMember() và không cho thêm null.

## 11.6 Bài 6 - Tổng hợp mini Order

### YÊU CẦU

Tạo Product, Customer, OrderItem, Order. Order có Customer; Order quản lý nhiều OrderItem; OrderItem tham chiếu Product; có CalculateTotal(); validation số lượng và giá.

## 11.7 Câu hỏi lý thuyết

13. Tại sao Car : Engine thường là thiết kế sai?
14. Aggregation và Composition khác nhau chủ yếu ở điểm nào?
15. “new object ở trong constructor” có phải lúc nào cũng đủ để kết luận composition không?
16. Vì sao IReadOnlyList<T> có thể tốt hơn public List<T> trong một aggregate?
17. Một quan hệ một-nhiều có nhất thiết là composition không?
18. Interface và composition giải quyết hai vấn đề khác nhau như thế nào?
19. Vì sao composition có thể giúp thay thế nhu cầu đa kế thừa class?

## Tóm tắt cần nhớ

### GHI NHỚ

- Làm bài 1-3 để luyện nhận diện từng quan hệ.
- Làm bài 4 để luyện tư duy mô hình.
- Làm bài 5-6 để kết nối quan hệ object với encapsulation.

**Một câu cần nhớ:** Nếu bạn giải thích được “vì sao chọn quan hệ này” chứ không chỉ viết được code, bạn đã hiểu bài.

## 12. Đáp án và hướng giải

### 12.1 Bài 1 - Composition

```
public class CPU
{
    public string Model { get; }
    public int CoreCount { get; }

    public CPU(string model, int coreCount)
    {
        if (string.IsNullOrEmpty(model))
            throw new ArgumentException(nameof(model));

        if (coreCount <= 0)
            throw new ArgumentException(nameof(coreCount));

        Model = model.Trim();
        CoreCount = coreCount;
    }
}

public class Computer
{
    public string Name { get; }
    public CPU CPU { get; }

    public Computer(
        string name,
        string cpuModel,
        int coreCount)
    {
        if (string.IsNullOrEmpty(name))
            throw new ArgumentException(nameof(name));

        Name = name.Trim();
        CPU = new CPU(cpuModel, coreCount);
    }
}
```

Computer tự tạo CPU. Người gọi chỉ cung cấp dữ liệu để Computer tạo thành phần của nó.

### 12.2 Bài 2 - Aggregation

```
public class Student
{
    public string Id { get; }
    public string FullName { get; }

    public Student(string id, string fullName)
    {
        Id = id;
        FullName = fullName;
    }
}

public class Course
{
    private readonly List<Student> _students = new();
    public IReadOnlyList<Student> Students => _students;

    public void AddStudent(Student student)
```

```

    {
        if (student is null)
            throw new ArgumentNullException(nameof(student));

        _students.Add(student);
    }
}

```

```

Student s1 = new Student("SV01", "An");
Student s2 = new Student("SV02", "Binh");

Course course = new Course();
course.AddStudent(s1);
course.AddStudent(s2);

```

Student được tạo độc lập và chỉ được Course tham chiếu/quản lý trong danh sách.

### 12.3 Bài 3 - Association

```

public class Patient
{
    public string Name { get; }

    public Patient(string name)
    {
        Name = name;
    }
}

public class Doctor
{
    public string Name { get; }

    public Doctor(string name)
    {
        Name = name;
    }

    public void Examine(Patient patient)
    {
        if (patient is null)
            throw new ArgumentNullException(nameof(patient));

        Console.WriteLine(
            $"Doctor {Name} examines {patient.Name}");
    }
}

```

### 12.4 Bài 4 - Phân loại

| Cặp          | Gợi ý đáp án            | Giải thích                                                          |
|--------------|-------------------------|---------------------------------------------------------------------|
| Dog - Animal | is-a                    | Dog là một Animal.                                                  |
| Car - Engine | Composition/Aggregation | Phụ thuộc Engine có độc lập và có thể thay thế/chuyển xe hay không. |

| Cặp                   | Gợi ý đáp án             | Giải thích                                               |
|-----------------------|--------------------------|----------------------------------------------------------|
| Department - Employee | Aggregation              | Employee thường tồn tại độc lập và có thể chuyển phòng.  |
| Doctor - Patient      | Association              | Hai object tương tác; không sở hữu vòng đời nhau.        |
| Order - OrderItem     | Composition              | OrderItem thường là phần cấu thành của Order.            |
| Invoice - IPrintable  | Interface implementation | Đây là khả năng/hành vi, không phải has-a.               |
| Student - Address     | Phụ thuộc nghiệp vụ      | Có thể aggregation/composition tùy cách quản lý Address. |

## 12.5 Bài 5 - Encapsulation collection

```

public class Member
{
    public string Name { get; }

    public Member(string name)
    {
        Name = name;
    }
}

public class Team
{
    private readonly List<Member> _members = new();

    public IReadOnlyList<Member> Members => _members;

    public void AddMember(Member member)
    {
        if (member is null)
            throw new ArgumentNullException(nameof(member));

        _members.Add(member);
    }
}

```

## 12.6 Bài 6 - Hướng giải mini Order

Bạn có thể dùng chính thiết kế ở Chương 9. Điều cần tự kiểm tra là: không public set bữa bãi, không cho quantity <= 0, Order không expose List<OrderItem> để bên ngoài Clear trực tiếp, và OrderItem không kế thừa Product.

## 12.7 Đáp án câu hỏi lý thuyết ngắn

**Car : Engine sai vì sao?** Vì Car không phải là một Engine. Quan hệ đúng thường là Car has-an Engine.

**Khác biệt chính Aggregation/Composition?** Mức độ sở hữu và sự độc lập vòng đời của object thành phần.

**new ở trong có đủ kết luận composition?** Không. Đó là dấu hiệu dễ thấy, nhưng cần xem bản chất ownership và vòng đời.

**Vì sao IReadOnlyList<T>?** Giảm khả năng code ngoài sửa collection tùy ý; class giữ quyền kiểm soát thay đổi.

**One-to-many có luôn là composition?** Không. Cardinality và loại quan hệ là hai vấn đề khác nhau.

**Interface khác composition?** Interface mô tả khả năng; composition mô tả object có/chứa object khác.

**Composition thay đa kế thừa thế nào?** Một class có thể chứa nhiều thành phần và triển khai nhiều interface thay vì kế thừa nhiều class.

## 13. Cheat sheet và checklist tự đánh giá

### 13.1 Bảng nhớ nhanh

| Khái niệm   | Câu hỏi cần hỏi           | Ví dụ                         |
|-------------|---------------------------|-------------------------------|
| Inheritance | B có phải là một A?       | Circle is-a Shape             |
| Association | A có tương tác/sử dụng B? | Doctor examines Patient       |
| Aggregation | A có B nhưng B độc lập?   | Department has Employees      |
| Composition | B là phần gắn chặt của A? | Order has OrderItems          |
| Interface   | Object có khả năng gì?    | Invoice implements IPrintable |

### 13.2 Công thức nhớ

is-a -> inheritance  
has-a -> object relationship

Association -> liên kết / sử dụng  
Aggregation -> has-a lỏng  
Composition -> has-a chặt

1 base class  
+ nhiều interface  
+ composition khi cần nhiều thành phần

### 13.3 Checklist trước khi chọn inheritance

- Tôi có thể nói “B là một A” không?
- B có thể được sử dụng ở mọi nơi cần A không?
- Tôi cần polymorphism qua A hay chỉ muốn dùng lại code?
- Nếu chỉ cần một thành phần/hành vi, composition hoặc interface có phù hợp hơn không?

### 13.4 Checklist phân biệt Aggregation và Composition

- Object thành phần có ý nghĩa độc lập không?
- Nó có được dùng bởi object khác không?
- Ai chịu trách nhiệm tạo/quản lý nó?
- Nó có thể được chuyển sang object chứa khác không?
- Nếu object chứa biến mất trong nghiệp vụ, object thành phần có còn ý nghĩa không?

### 13.5 Tự đánh giá sau bài

| Mức       | Bạn làm được gì?                               |
|-----------|------------------------------------------------|
| Chưa chắc | Nhớ tên khái niệm nhưng chưa phân loại được ví |

| Mức | Bạn làm được gì?                                                       |
|-----|------------------------------------------------------------------------|
|     | dụ mới.                                                                |
| Đạt | Phân biệt được is-a/has-a và viết được 3 bài cơ bản.                   |
| Khá | Giải thích được ownership/vòng đời và encapsulate collection.          |
| Tốt | Tự thiết kế mô hình Order/Customer/Product và bảo vệ lựa chọn quan hệ. |

## KẾT LUẬN

Sau phần này, bạn không chỉ biết tạo class kế thừa nhau mà đã bắt đầu thiết kế cách các object phối hợp. Đây là bước chuyển quan trọng từ “biết cú pháp OOP” sang “biết mô hình hóa đối tượng”.

## 13.6 Nội dung kế tiếp trong lộ trình

Sau khi hoàn thành bài tập của phần này, bước tiếp theo là Collection & Generic: hiểu sâu List<T>, Dictionary<TKey,TValue>, HashSet<T>, Queue<T>, Stack<T> và cách Generic giúp viết code tái sử dụng an toàn kiểu dữ liệu.